

xStore — Refund Methodology: Flows & Required Fixes

End-to-end refund flows with concrete build requirements for Backend, Mobile, and Admin Dashboard · prepared 2026-08-09

This document turns the case catalogues in **04 — Vendor Commission Wallet & Settlement** and **05 — Buyer Refund Methodology** into buildable flows. For each flow: the step-by-step sequence across every actor, then exactly what needs to be built on **Backend**, **Mobile** (buyer / vendor / courier apps), and the **Admin Dashboard** — grounded in what already exists today, not invented from scratch.

Dependency note: there is currently no Orders/Checkout backend at all — every flow below assumes that gets built first (order + order-item persistence, status transitions). Everything here is the refund layer that sits on top of it. The Admin Dashboard side references its existing prototype (`docs_business/admin-dashboard/src/app.js`), which already has a Disputes screen with Refund / Partial / Reject buttons — today those are UI mockups only (a toast message, no real logic). This document specifies how to wire that screen up for real, rather than building a second one.

Actor legend used in every flow

[BUYER] [VENDOR] [COURIER] [BACKEND] [ADMIN]

Flow 1 — Self-Delivery Return & Refund

Applies when the order was fulfilled by the vendor directly (no courierId) — the vendor holds the cash, so this is a vendor-trust flow, not a platform-money flow.

1. **[BUYER]** Opens the delivered order in Order Detail, taps “Request Return / Refund” (shown only within the return window on self-delivery orders — see §7 for the recommended window).
2. **[BUYER]** Selects the item(s) to return, a reason (defective / wrong item / not as described / changed mind / other), and optionally attaches a photo.
3. **[BACKEND]** Creates a ReturnRequest (status: requested) linked to the order-item(s); notifies the vendor.
4. **[VENDOR]** Reviews the request in a Return Requests queue. Approves (status: approved) or rejects with a required reason (status: rejected — buyer can escalate, see Flow 4).
5. **[VENDOR]** Coordinates directly with the buyer to collect the item and hand back cash, then taps “Mark as Refunded” (status: refunded).
6. **[BACKEND]** On status → refunded, reverses the commission for the affected order-item(s) per the companion Commission Wallet document, updating the vendor's wallet balance.
7. **[BUYER]** Sees the return status update to Refunded. If the buyer disputes ever receiving the cash despite the vendor's claim, they can escalate (Flow 4).

BACKEND — required changes

- New **ReturnRequest** entity: orderId, orderItemId(s), reason, photoUrl (optional), status (requested / approved / rejected / refunded / escalated), timestamps, requestedBy / resolvedBy.
- Endpoints: POST /orders/{id}/return-requests, PATCH /return-requests/{id}/approve, /reject, /mark-refunded.
- Commission-reversal hook on mark-refunded: reverse commissionAmount for the affected order-item(s), same mechanism as the accrual state machine in doc 04 §1.
- Server-side return-window validation (not just a UI check) — a single configurable constant, same pattern as the existing commission thresholds.

MOBILE — Buyer app

- order_detail_screen.dart: “Request Return / Refund” action on delivered, in-window, self-delivery orders.
- New ReturnRequestScreen: item picker for multi-item orders, reason dropdown, photo attach.
- Return status surfaced on order detail: Requested → Approved → Refunded, or Rejected with an Escalate action.

MOBILE — Vendor app

- New “Return Requests” list (or a filter/tab on vendor_orders_screen.dart) with a badge count.
- vendor_order_detail_screen.dart: Approve / Reject actions (reject requires a reason); “Mark as Refunded” action once approved.

ADMIN DASHBOARD

- No change required for the happy path — the platform isn't a party to a self-delivery refund unless it's escalated (Flow 4).

Flow 2 — Courier-Delivery Return & Refund

Applies when the order was fulfilled by a platform courier (courierId set). The platform collected the COD cash and may already have paid the vendor's net share — this is the one flow where the platform's own money is at risk, not just the vendor's.

1. **[BUYER]** Same entry point as Flow 1 (Order Detail → “Request Return / Refund”); the app detects courierId is set and routes down this path instead.
2. **[BACKEND]** Creates the ReturnRequest with fulfillmentChannel: courier, and checks vendorPayoutStatus for that order — has the platform already paid the vendor their net share in a courier cash-settlement cycle?
3. **[ADMIN]** Because real platform cash is on the line, vendor self-approval is skipped — the request routes straight to the Admin Dashboard's Disputes/Refunds queue for owner/ops review.
4. **[ADMIN]** Reviews the case (reason, photo, order value, current payout state) and approves or rejects.
5. **[BACKEND]** If approved and the vendor has **not yet been paid** net for this order: excludes it from the next payout batch and refunds the buyer directly — no clawback needed. If the vendor **has already been paid**: creates a clawback entry on the vendor's existing wallet ledger (a new reason code alongside commission-due, same balance, same warn/pause enforcement) — the vendor now owes the platform that net amount back.
6. **[ADMIN]** If the item needs to physically come back, assigns a courier to a reverse-pickup job from the same Delivery/Orders screen already used for forward deliveries (same assign-courier drawer, new job type).
7. **[COURIER]** Sees the reverse-pickup job in the same list pattern as normal deliveries, collects the item from the buyer, confirms pickup.
8. **[BACKEND]** Pays the buyer's refund (channel per §7) and closes the ReturnRequest as refunded.
9. **[VENDOR]** Receives a notification only — no action required unless a clawback landed on their wallet, where it appears exactly like commission debt does today.

BACKEND — required changes

- ReturnRequest.fulfillmentChannel (self / courier) + a vendorPayoutStatus lookup at request-creation time.
- Admin-approval gate for courier-channel requests — bypasses the vendor self-approval used in Flow 1.
- Extend the vendor wallet ledger to carry a second entry type — **payout clawback** — alongside the existing commission-due entries: same balance, same warn/pause thresholds, different origin reason so the vendor's statement can explain *why* they owe more.
- New reverse-pickup job type (or a flag on the existing courier-order model) so it reuses the existing courier-assignment and cash-wallet machinery instead of a parallel system.
- Buyer-refund-payment record — however the channel is chosen (§7) — so it's auditable who paid the buyer back and when.

MOBILE — Buyer app

- Same ReturnRequestScreen as Flow 1, but status copy differs for courier-channel orders: “Under platform review” instead of “waiting on vendor.”

MOBILE — Vendor app

- Read-only status/notification on the affected order (“A courier-delivered order is being refunded — no action needed”).
- Wallet/earnings view surfaces the clawback line item if one lands, using the same due/settled visual pattern already built for commission.

MOBILE — Courier app

- New “Return pickups” section on courier_deliveries_screen.dart (or a filter on the existing active-run list), reusing the pickup/drop-off card pattern already built for forward deliveries.

ADMIN DASHBOARD — required changes

- The existing **Disputes** screen (`disputes()` / `disputeDrawer()` in the current prototype) already has Refund buyer / Partial / Reject buttons — today they only show a toast with no real backend call. Wire them to the `ReturnRequest/refund` endpoints above.
- Add a fulfillment-channel badge to each dispute row/drawer (self-delivery vs. “ Delivered by xStore”) so ops can see at a glance which cases are pure arbitration vs. which involve real platform cash.
- Add a vendor-ledger-impact preview inside the dispute drawer for courier-channel cases (e.g. “Approving this creates a EGP 480 clawback on Alexandria Beauty Bar’s wallet” or “will be excluded from their next payout”) so admin isn’t approving blind.
- Reuse the existing Orders screen’s delivery/assign-courier pattern (`deliveryCell()`) to assign a reverse-pickup courier directly from the dispute drawer.

This is the highest-priority flow in the whole document — it’s the only one where the platform, not the vendor, is financially exposed. Recommend building this before courier delivery scales past the pilot.

Flow 3 — Vendor-Initiated Partial Refund / Price Adjustment

Applies when vendor and buyer agree to a partial refund without the item coming back — e.g. a minor cosmetic defect where the buyer keeps the item.

1. **[VENDOR]** On the order detail screen, taps “Issue Partial Refund,” enters an amount (capped at the order-item total) and a reason.
2. **[BACKEND]** Creates a lightweight OrderAdjustment record — distinct from a full ReturnRequest, since no item comes back and (by default, see §7) commission is not reversed, because the sale still fully occurred.
3. **[VENDOR]** Refunds the buyer directly in cash — same trust-based mechanic as Flow 1's self-delivery case.
4. **[BACKEND]** For courier-delivered orders, the adjustment amount is either excluded from the vendor's next payout or becomes a clawback line — reusing Flow 2's ledger machinery rather than inventing a new one.

BACKEND — required changes

- New lightweight OrderAdjustment (amount, reason, orderItemId) — no return-item state machine needed.
- Reuse Flow 2's clawback / exclude-from-payout logic for the courier-channel case; no new ledger mechanism.

MOBILE — Vendor app

- “Issue Partial Refund” action + amount/reason form on `vendor_order_detail_screen.dart`.

ADMIN DASHBOARD

- No new screen — courier-channel adjustments surface through the same wallet clawback visibility built for Flow 2.

Flow 4 — Refund Dispute / Fraud Escalation

Applies when a vendor rejects a return the buyer believes is legitimate, a buyer claims non-receipt of a cash refund the vendor claims they paid, or evidence suggests fraud on either side.

1. **[BUYER]** On a rejected or stalled return, taps “Escalate to xStore” — required to attach evidence (item photo, delivery confirmation, etc.).
2. **[BACKEND]** Marks the ReturnRequest escalated and creates/links a dispute case; notifies Admin.
3. **[ADMIN]** Case appears in the same Disputes queue used for Flow 2's courier-channel cases — one queue, not two — with buyer and vendor evidence attached.
4. **[ADMIN]** Decides: Refund buyer / Partial / Reject claim — the exact three actions already prototyped in the dashboard's dispute drawer.
5. **[BACKEND]** Executes the decision, routing to the correct refund/commission logic per the order's fulfillment channel (reuses Flow 1/2, doesn't duplicate it); records the resolution against both the buyer's and vendor's history.
6. **[ADMIN]** Case closes; both parties notified.

BACKEND — required changes

- Evidence-attachment endpoint (buyer and vendor can both attach to a dispute).
- Dispute-resolution endpoint that fans out to the correct refund/commission logic per channel — reuses Flow 1/2 rather than duplicating it.
- Per-vendor dispute-history aggregate (count / rate), queryable — not required for v1 UI, but the schema should support it later without a migration, for future trust scoring.

MOBILE — Buyer app

- “Escalate to xStore” action + evidence attach, shown once a return is rejected or stalls past the SLA (\$7).

MOBILE — Vendor app

- “Reject with evidence” requires an attachment, not just free text — raises the bar above a one-tap dismissal.

ADMIN DASHBOARD

- No new screen — this is the same Disputes queue as Flow 2, just a different entry path (buyer escalation vs. automatic courier-channel routing). The channel badge and ledger-impact preview built for Flow 2 apply here too.

Consolidated build checklist, by team

Deduplicated across all four flows — use this as the actual sprint-planning input.

BACKEND

- ReturnRequest entity + status state machine (requested / approved / rejected / refunded / escalated).
- OrderAdjustment entity for no-return partial refunds.
- fulfillmentChannel + vendorPayoutStatus awareness on every return/adjustment.
- Vendor wallet ledger extended with a payout-clawback entry type alongside commission-due.
- Reverse-pickup job type reusing the courier-assignment and cash-wallet system.
- Admin-approval gate for courier-channel requests.
- Evidence-attachment + dispute-resolution endpoints.
- Commission-reversal hook wired to ReturnRequest.mark-refunded (ties into doc 04's accrual state machine).
- Buyer-refund-payment record for auditability, whatever channel §7 lands on.

MOBILE

- **Buyer app:** Request Return/Refund entry point, ReturnRequestScreen, status tracking, Escalate action.
- **Vendor app:** Return Requests queue, Approve/Reject/Mark-Refunded actions, Issue Partial Refund action, clawback line item on the wallet screen.
- **Courier app:** Return pickups section reusing the existing delivery card pattern.

ADMIN DASHBOARD

- Wire the existing Disputes screen's Refund / Partial / Reject buttons to real endpoints (currently mock-only).
- Fulfillment-channel badge on every dispute row/drawer.
- Vendor-ledger-impact preview before approving a courier-channel refund.
- Reverse-pickup courier assignment, reusing the existing Orders delivery/assign-courier pattern.

Recommended build order

Everything here is blocked on the Orders backend existing at all (order + order-item persistence, status transitions) — that's the real critical path, not the refund logic itself. Once that exists:

- 1. Backend: ReturnRequest + Dispute schema and endpoints — unblocks every flow below.
- 2. Flow 1 (self-delivery) end to end, mobile buyer + vendor UI — ships standalone, no admin dependency.
- 3. Flow 4 (escalation) alongside Flow 1, not as an afterthought — a reject action without an escalation path is a dead end for the buyer from day one.
- 4. Backend: wallet clawback extension + admin-approval gating — required before Flow 2 can go live at all.
- 5. Mobile: courier reverse-pickup UI.
- 6. Admin: wire the existing Disputes screen to real endpoints + channel badge + ledger preview.
- 7. Flow 3 (partial refund, no return) — additive, can ship any time after step 1.

Owner decisions — recommended defaults

These are policy calls, not engineering ones. Recommended defaults below so backend isn't blocked — flag any you'd rather change before this goes into a sprint.

Decision	Recommended default	Why
Courier-held-cash refund mechanism	Net against the vendor's next payout if not yet paid; explicit clawback ledger entry (new reason code, same wallet) if already paid.	Avoids inventing a second money-movement system — reuses the wallet's existing warn/pause machinery.
Delivery fee refund scope	Refund the delivery fee only when the return reason is vendor-fault (defective / wrong item / not as described); not for “changed mind.”	Keeps the incentive aligned — the platform/vendor shouldn't eat a fee for a return that wasn't their error.
Return-pickup ownership	Courier network handles reverse pickup for courier-delivered orders (reuses existing infra); self-delivery returns are the vendor's own logistics, matching how they handled the forward delivery.	Symmetry with how the order was originally fulfilled — no new operational model needed.
Refund SLA	Vendor must approve/reject within 48 hours of a request or it auto-escalates to Admin; once approved, refund within 24 hours before the buyer can escalate non-receipt.	Makes the SLA enforceable (auto-escalation) rather than just implied, without being so tight it's unrealistic for cash logistics.
Does a partial refund without a return reduce commission?	No, by default — the item stays sold and delivered; only full/partial returns (item comes back) reverse commission.	The sale still fully occurred; treating every discount as a commission event would be a larger schema and policy change for a v1 that doesn't need it.
Refund payment channel for the courier-held-cash case	Mobile wallet (Vodafone Cash / InstaPay) as primary, in-person courier cash hand-off as fallback.	Mirrors the same channel question already open for vendor-to-platform settlement in doc 04 §3 — worth deciding both together.

Companion documents: **04 — Vendor Commission Wallet & Settlement** (the commission-reversal side of a return) and **05 — Buyer Refund Methodology** (the case catalogue this document turns into flows).